

AI Engineer

Technical Assessment

Overview

This is a take-home technical assessment for the AI Engineer role. It is designed to be completed in 2–3 hours, with a maximum of 4 hours.

You will design a **recommendation engine for a high-frequency (HF) iGaming sportsbook**, with both *online* (real-time) and *offline* (scheduled batch) recommendation paths. We are as interested in how you reason about the problem as in what you produce.

What we are assessing

This is not an implementation exercise. You are not expected to train a model, build a working pipeline, or ship running code. We are assessing your applied ML and recommender-system judgement, and the reasoning behind it. Concretely, we evaluate:

- Your architectural thinking for the online/offline split and how they compose
- Your handling of what makes HF sportsbook data hard (freshness, non-stationarity, regulatory constraints)
- Your evaluation thinking — offline, online, and the feedback-loop pathologies specific to recommenders
- How clearly you communicate decisions and trade-offs
- How you work with AI coding agents, and how the submission would be handed off

Time guidance

We expect roughly 2–3 hours of focused work. **Do not exceed 4 hours.** We would rather see a well-reasoned, partially-scoped design with a clear “what I would do with more time” section than an exhausted attempt to cover everything. Scoping decisions are themselves part of what we assess.

Scenario

You are designing a recommendation engine for a B2B iGaming sportsbook platform. The platform receives high-frequency event streams: pre-match and in-play odds updates (fast-moving), user activity (odds views, bet slips, placed bets, cash-outs, session context), and a large, time-decaying catalog of markets across many sports (matches expire; new ones appear continuously).

The product goal is to recommend *relevant markets and bets* to each user, surfaced across multiple placements (homepage carousel, in-play sidebar, post-bet suggestions). The business cares about user engagement and platform value, but not at any cost — recommendations must respect **Responsible Gambling** constraints and per-jurisdiction eligibility.

How we mean “online” and “offline”

This assessment uses these terms in a specific way:

- **Offline recommendations** are produced by a scheduled batch process. The job runs on a cadence (nightly, hourly, whatever you justify) and pre-computes recommendation itemsets per user (or per segment) that are stored and served from a lookup at request time. Cheap to serve, but stale by the time they are shown.
- **Online recommendations** are produced at request time (or triggered by live events). They can reflect the current state of the sportsbook — live odds, in-play availability, what the user did seconds ago — at the cost of a tighter latency budget and more expensive serving.

A meaningful part of the assessment is how you make these two work together rather than treating them as independent systems.

Your task

Produce a design document (and supporting code stubs) describing the recommendation engine, its offline and online paths, and how they compose.

In scope

- **The overall architecture** — the components involved (candidate generation, ranking, re-ranking, feature store, serving, batch job, etc.), what data flows where, and how the online and offline paths compose (does online filter/re-rank an offline candidate pool? does it fall back? does it override?).

- **The offline path** — what job runs, on what cadence and why, what data it consumes, what it produces (the shape of an “itemset”), how it is stored and looked up, and how staleness is handled.
- **The online path** — what happens at request time (or on which event triggers), what features/signals are available, what the latency budget is, and how live sportsbook state (odds, in-play, session) is incorporated.
- **Modelling choices** — candidate generation approach (heuristic, embedding-based, two-tower, etc.), ranking approach, cold-start handling, and how heterogeneous signals (bet history, odds views, cash-outs, dwell) are used. Simple, well-justified choices are strongly preferred over ambitious ones you cannot defend.
- **Responsible Gambling and eligibility** — how RG signals and jurisdictional rules constrain what may be recommended, and where in the pipeline those constraints are applied.
- **Evaluation** — how you would evaluate this offline (metrics, holdouts, counterfactual considerations) and online (A/B, guardrail metrics), and what “success” means beyond raw click-through.

Out of scope

You do not need to: train a real model, produce a runnable end-to-end system, build ingestion or data pipelines, or write production-grade code. Assume upstream event streams and a warehouse exist and are queryable.

What “considered but not built” should cover

In your design document, address (even if not in code): the offline/online contract (what shape the itemsets take and how the online layer consumes/overrides them), freshness and staleness handling, feedback loops and the pathologies specific to a sportsbook recommender (chasing losses, popularity bias, RG risks), monitoring and drift detection, and what you deliberately scoped out and why.

Required deliverables

Submit a Git repository (or a zip of one) containing the following. The **design document is the highest-signal artifact** — invest in it.

1. Design document

A written document (Markdown, PDF, or similar) covering, in your own words:

- **Architecture overview** — the components and how they fit together; a diagram is strongly encouraged (hand-drawn, ASCII, or a tool of your choice).
- **Offline path** — job design, cadence, inputs, itemset shape, storage, and staleness handling.
- **Online path** — request-time flow, latency budget, features used, and how live sportsbook state is incorporated.
- **Offline/online composition** — the contract between them; how online consumes, filters, re-ranks, or overrides offline output; fallback behaviour.
- **Modelling choices** — candidate generation, ranking, cold-start, and use of heterogeneous signals; the alternatives you rejected and why.
- **Responsible Gambling & eligibility** — how these constraints are enforced and where in the pipeline.
- **Evaluation** — offline metrics and methodology, online experimentation, guardrail metrics, and monitoring for drift and feedback-loop pathologies.
- **Scoped out & next steps** — what you deliberately did not cover, and what you would do with more time.

2. Code stubs

Include a small amount of code that *illustrates the design* rather than implements it. Aim for legibility, not completeness. Good examples of what to include:

- A schema/dataclass for the offline itemset (what a stored recommendation entry looks like).
- A function signature and docstring for the online ranker/re-ranker (inputs, outputs, latency budget, side-effect boundaries).
- A sketch of the feature contract shared between offline training and online serving.
- A short pseudocode block or notebook cell showing the online request-time flow end to end.

Do not try to build a runnable pipeline or train a model. Code that runs is not worth more than code that clearly conveys the design.

3. Agent-readiness

This role works with AI coding agents day to day, so we want to see how you do too. This part is **deliberately lightweight — it is not a second build**. Provide both of the following:

- **A short “How I used AI agents” note (in the design document):** which agent(s) you used, what you delegated versus did yourself, and at least one example where the agent got something wrong and how you caught and corrected it. We value an honest, critical account over a polished one.
- **Evidence the submission is ready for an agent to work in:** for example an agent instructions/conventions file (CLAUDE.md, AGENTS.md, or equivalent) describing repo conventions and how to validate the stubs; a navigable structure and clear README pointing to the design document; and explicit guardrails around anything sensitive (data assumptions, PII, RG constraints). You need not do all of these — show that you understand what makes a repo safe and legible for an agent to operate in.

4. A short “why this design” note

A few sentences on the **single biggest design decision** you made, what alternatives you rejected, and what you would want to validate first if you were building this for real. This is a scored part of the deliverable.